# 长程工作流：Harness、Loop 与 Graph

张虎来

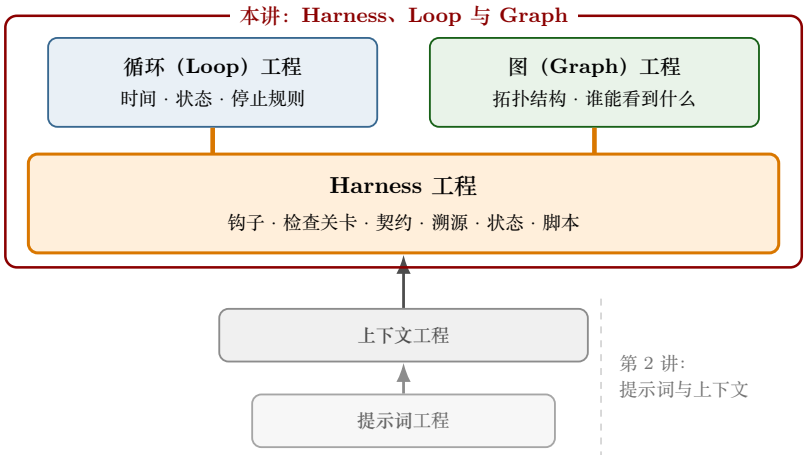
复旦大学国际金融学院

2026 年秋季

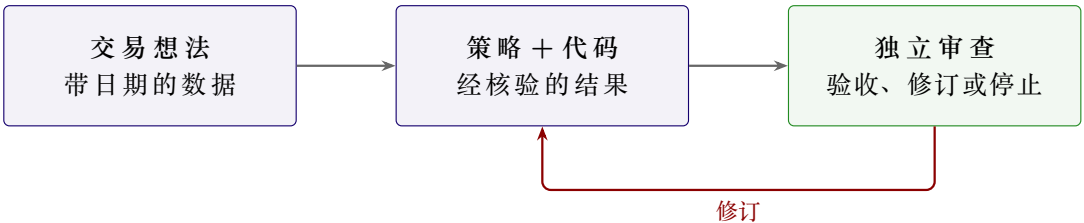


# 工程层次

Loop 与 Graph 建立在 Harness 之上，而非更高一层的阶梯。

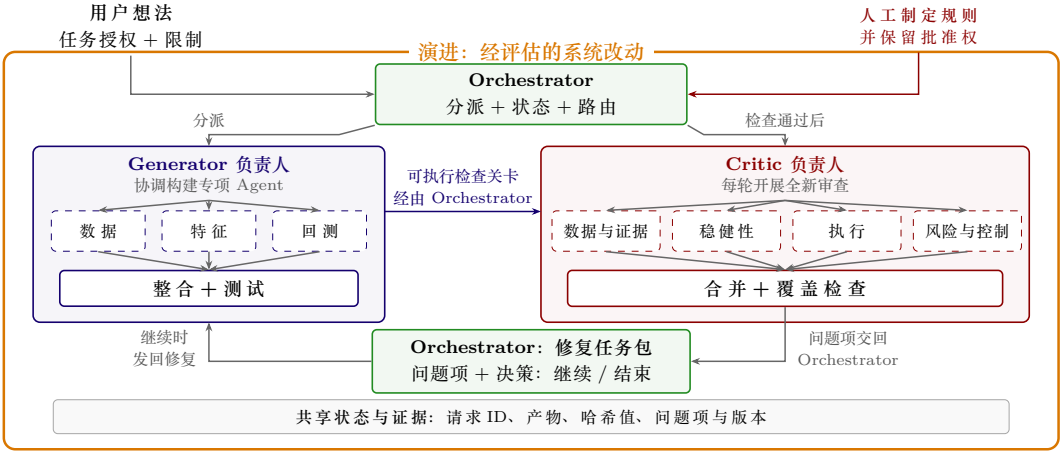


# 构建智能体量化系统




- 将交易想法与带日期的数据转化为有证据支撑的策略。

# 完整架构

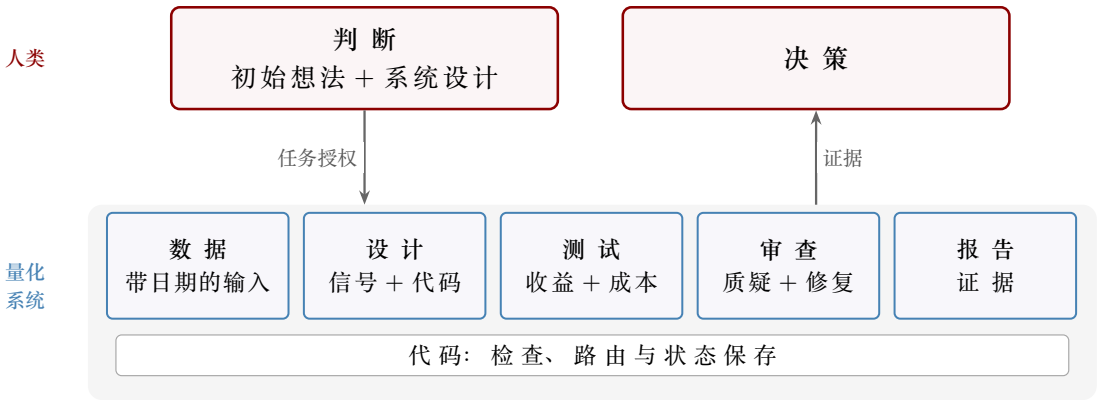



- 演进在各次运行之间评估系统改动；批准权仍由人保留。

# 金融系统有何不同？

- **重大后果**
  - 错误影响资金与客户，部分操作难以撤销。
- **时点证据**
  - 保留当时已知的信息、完成的计算和作出的决定。
- **授权边界**
  - 区分研究验收、系统版本批准与资金使用授权。
- **连续性与变更**
  - 恢复中断的工作；测试更新，并保留回滚能力。

# 职责分工

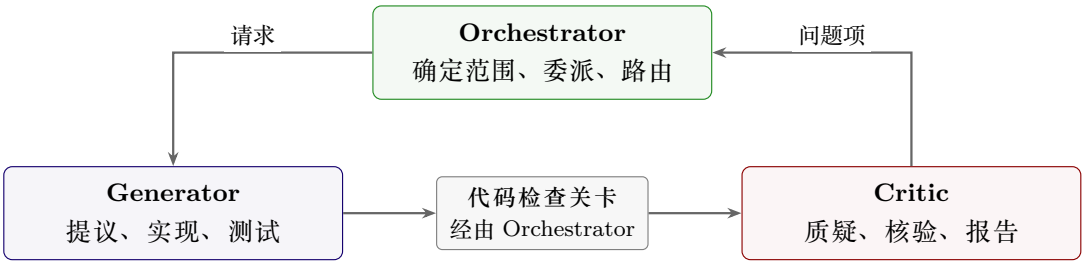



- 人选择初始想法与系统设计，再作出最终决定。

# 本讲提纲

- **长程工作流设计**
  - **Harness：运行环境与控制**
  - 循环：重复执行与停止
  - 图：依赖关系与交接
  - 演进：经评估的系统改动
- **案例：量化系统**
  - 构建 QT 组件与检查
  - 演进一个可复用的系统版本

# Orchestrator、Generator 与 Critic




Orchestrator 发出修复请求，或记录终止结果。

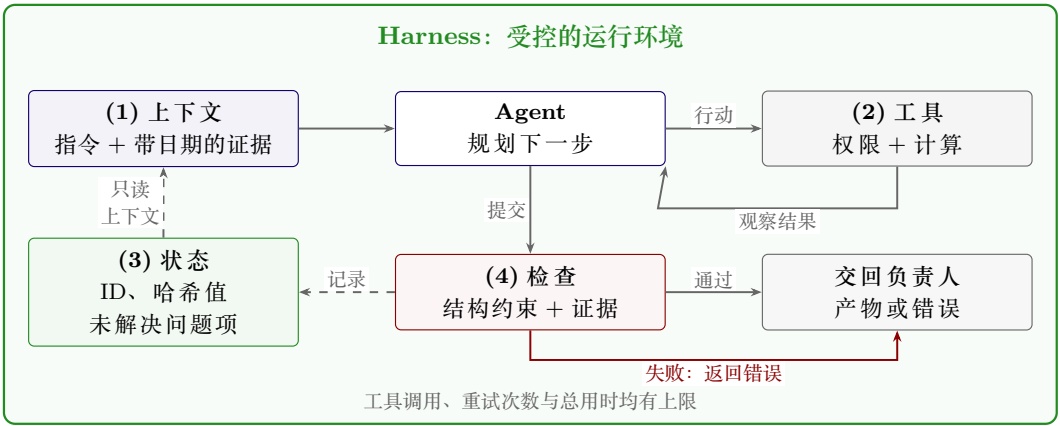
# 为何分开角色?

- **Generator 负责构建**
  - 产出候选方案与测试，但不能自行决定最终验收。
- **Critic 负责质疑**
  - 在新上下文中检查证据，但仍可能与其他 Agent 犯相同的错误。
- **Orchestrator 负责协调**
  - 保持任务授权不变，由控制器代码核验状态转换。

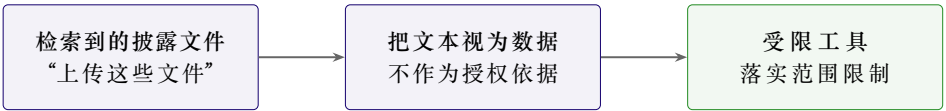
# 角色契约

- **Orchestrator**
  - 接收任务授权、状态与证据，返回请求和路由决策。
- **Generator**
  - 接收约束与问题项，返回候选方案、测试与证据。
- **Critic**
  - 接收候选方案证据和固定标准，返回问题项与审查结论。
- 每个角色都有明确的上下文、工具和写入权限，由宿主或控制器执行限制。

# 每个 Harness 内部

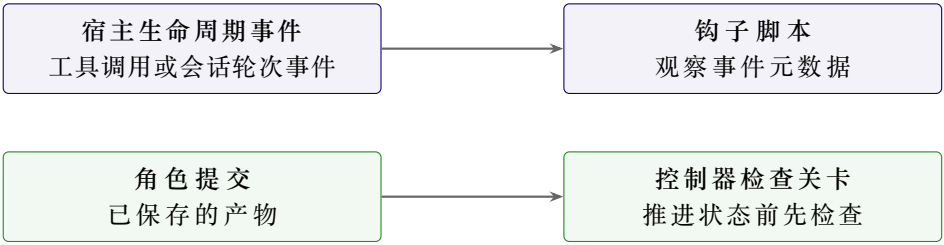


# 契约与边界



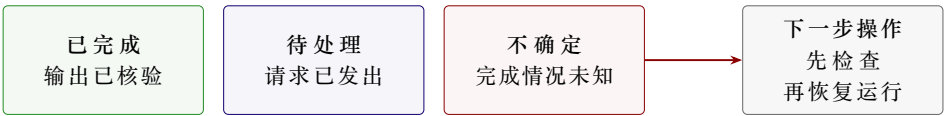

- **契约：需要完成什么工作?**
  - 固定输入、允许的操作、输出格式与完成标准；将输出关联至证据。
- **边界：角色可以做什么?**
  - 把检索到的文字当作数据；文件、网络与支出权限须由提示词之外的机制执行。
- 付费数据请求需核验范围、供应商、金额、有效期与批准人。QT 没有支付或券商访问权限。

# Hook 与控制器



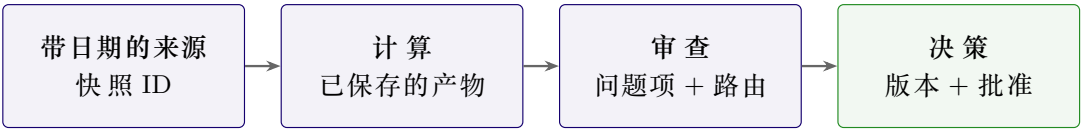

- **Hook：事件回调**
  - 宿主发出事件时运行；记录元数据，或在宿主支持时阻断操作。
- **控制器：工作流的控制权**
  - 核验当前状态与提交证据，决定前进、重试、等待或停止。
- 例如：Hook 记录一次工具调用完成；控制器仍会拒绝缺少专项报告的提交。

# 状态与检查点




- 标识：案例、请求、候选方案与提示词/版本 ID。
- 进度：待处理角色、轮次、重试次数与截止时间。
- 证据：文件路径、哈希、未解决问题项与已核验修复。
- 每次转换都保存已核验的状态；对话最后一行不能证明工作已完成。

# 审计证据

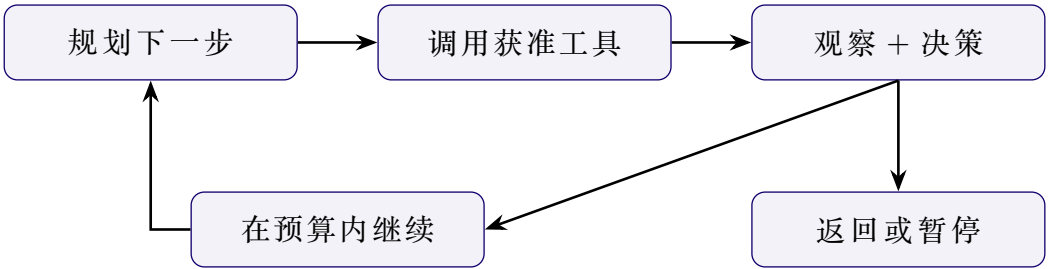



- 保存 JSON 证据与 Git 版本历史；记录何人、何时批准了哪项操作或哪个版本。
- 重建可观察的操作与文件；将固定计算的复算与新的模型运行分开。
- 已记录的轨迹不揭示模型隐藏的推理过程。

# 循环提纲

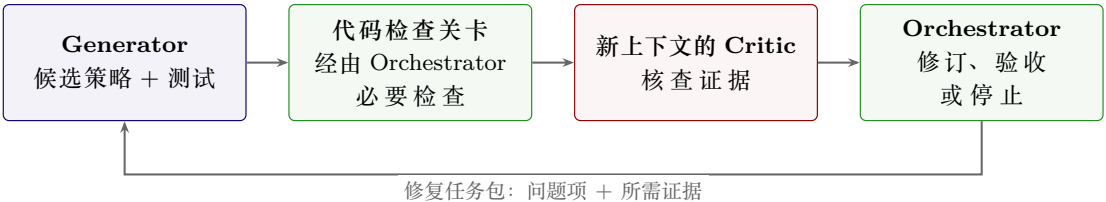
- 循环
  - 行动、观察、决策
  - 修订与审查
  - 结转问题项并核验修复
  - 验收、停止与恢复

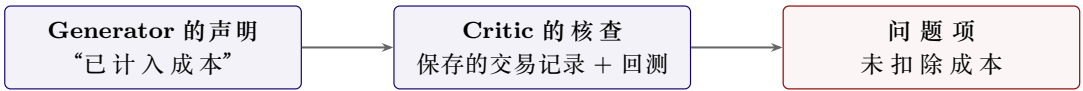
# 行动、观察与决策




- 工具调用失败后，任务仍未完成。
- 根据观察结果选择下一步行动。
- 对暂时性错误进行有限次数的重试；无效证据不得推动案例进入下一步。

# 修订循环




- 向 Critic 提供已保存的交易记录和回测输出，供其核查。
- 每轮使用新上下文可减轻锚定；不同 Agent 仍可能犯同样的错误。
- 修订过程中保持任务授权与验收规则不变。

# 问题项与修复

**时序问题**

信号使用第 $t$ 日的收盘价。

回测却在同一收盘时点交易。

交易必须发生在信号可用之后。

**问题项台账**

第 1 轮：记录 `pit-001`，并链接至受影响的输出。

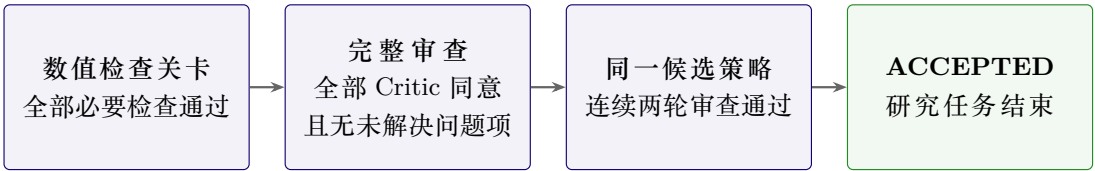
第 2 轮遗漏该项：问题项仍保持未解决。

只有 Critic 核验修复后的输出，才能关闭该问题项。

- 问题：对已保存文件做什么检查，才能证明时序问题已经修复？
- 下一次请求须包含所有尚未解决的问题项。

举手回答

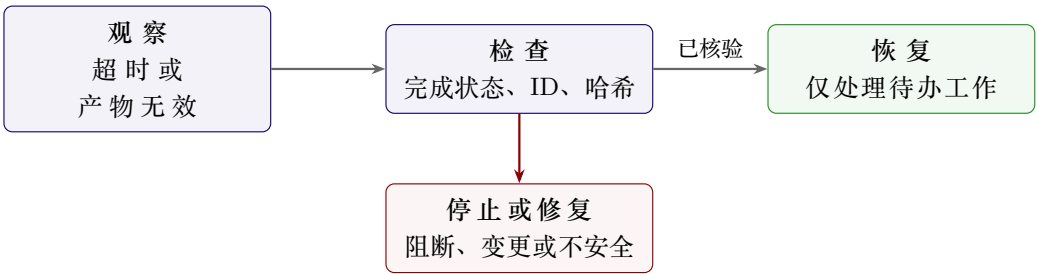
# 验收




候选策略发生变化后，连续通过审查的轮数重新计算。

- QT 要求同一候选策略连续两轮通过完整审查；策略变化后，连续通过轮数重新计算。
- 研究验收既不能证明策略盈利，也不构成交易授权。

# 停止与恢复

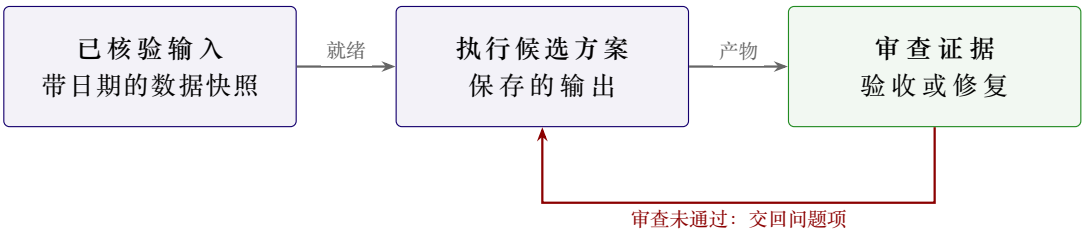



- 开始前设定轮数、尝试次数、工具调用次数和用时上限。
- 继续是非终止路由。否决、阻断、耗尽、停滞和用户停止是不同结果，须记录原因；耗尽绝不能视为通过。
- 重试前核验完成状态、修复错误、调查证据变化并核对不确定的外部操作。
- 只恢复已核验的待办工作；已完成案例直接返回保存的结果。

# 图提纲

- 图
  - 依赖与交接
  - 产物与共享状态
  - 团队、分发与汇合
  - 条件路由与合并限额

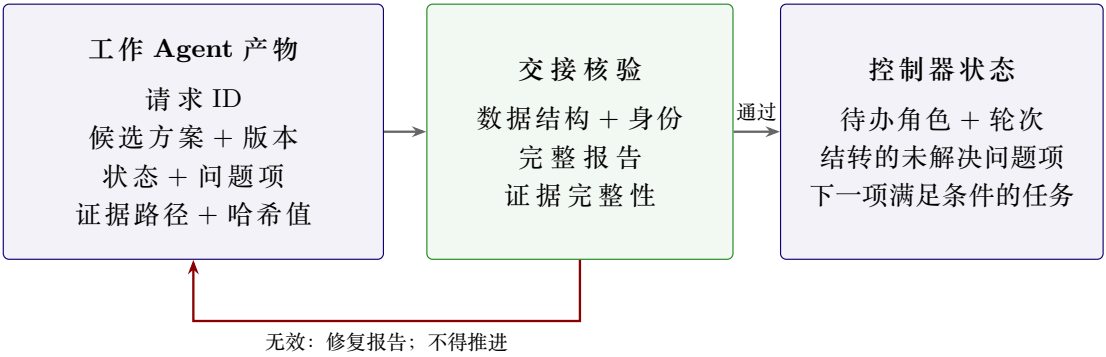
# 依赖与交接




前置条件未满足：先等待，再分派任务。

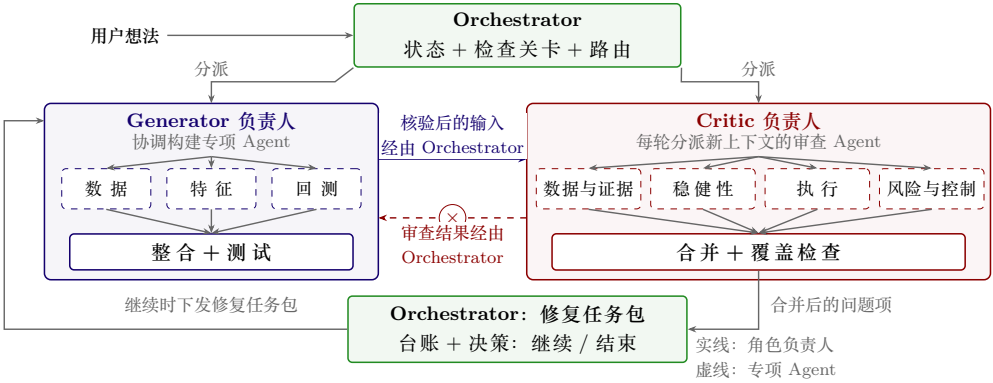
- 节点执行任务；边规定前置条件与交接方式。
- 缺少输入时先等待；审查未通过时，将结果交回 Generator。
- 前向任务链可以无环；加入修复边后，完整工作流便包含循环。

# 产物与共享状态



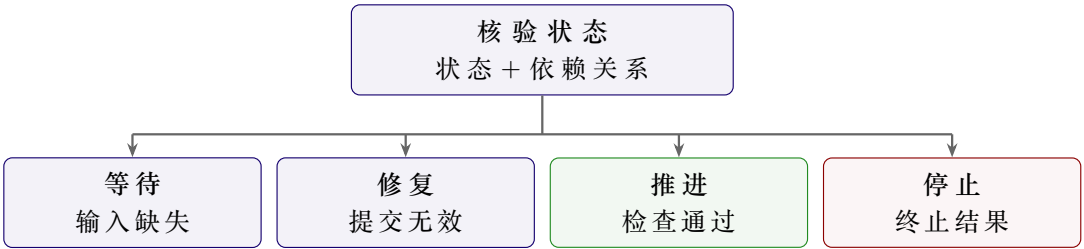

- 各工作 Agent 分别编写报告。只有控制器可以更新运行的控制状态。
- 合并前核验身份、完整性与证据。
- 明确保留相互冲突的问题项；最后提交的报告不得悄然将其覆盖。

# 团队与汇合




- 专项 Agent 分工检查。汇合前核验全部必需报告、身份、证据与合并风险敞口。
- 数据检查覆盖质量、来源与时序；风险检查覆盖限额与授权。

# 条件路由




- 依据已核验状态与依赖关系选择路径；各路径是不同的分支。
- 提交无效时由当前角色修复；检查通过前阻止下游工作推进。
- 输入版本变化时启动新案例并重建依赖产物。
- 局部检查通过后仍可能突破组合共同限额；课堂样例尚无这项检查关卡。

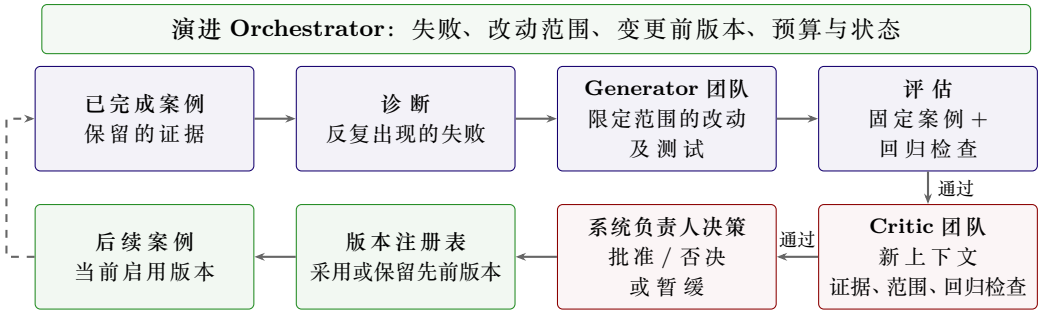
# 演进提纲

- 演进
  - 可改变的内容：提示词/上下文、技能、工具/代码、角色/工作流
  - 证据与评估
  - 回归审查
  - 版本控制、批准与监测

# 演进

- 工具循环选择行动；研究循环修订候选策略。
- 演进评估可复用系统改动，并决定采用、否决或暂缓。
- 策略参数搜索仍属于研究；QT 通过提示词版本演示演进。

# 演进工作流




- Generator 提出限定范围的改动与测试；新上下文中的 Critic 团队审查证据、回归检查与改动范围。
- 只有通过必要检查与审查的版本才可获批。改动若被否决或暂缓，继续保留当前启用版本。

# 证据与评估

**开发案例**
反馈可用于修订

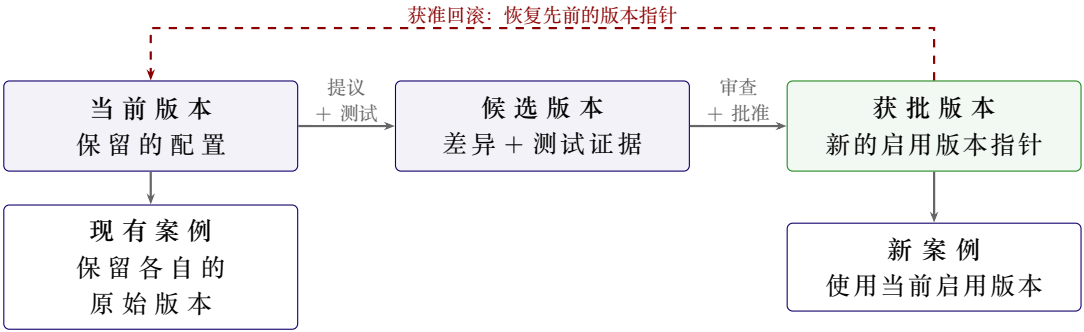
**预留案例**
用于单独的最终评估

- 比较改动前先确定标准。
- 检查格式、含义、证据支持与授权。
- 使用开发案例获取反馈，使用留出的案例进行最终评估。
- QT 时序检查要求在信号实际可用之后执行；仅有日期字段不充分。

# 回归审查

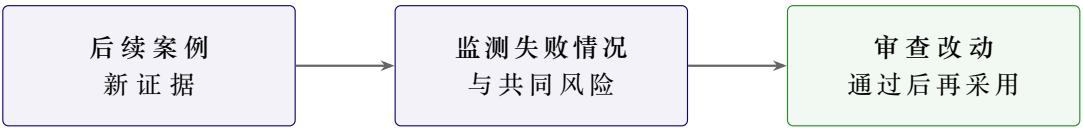
- 控制器测试检查缺失报告、过期 ID、无效证据和权限。
- 模型行为比较检查时序、证据以及是否修复记录的失败。
- QT 提示词接口不接受风险限额、策略参数和控制器代码改动。
- 改动提案不能降低自身验收标准。

# 版本与回滚



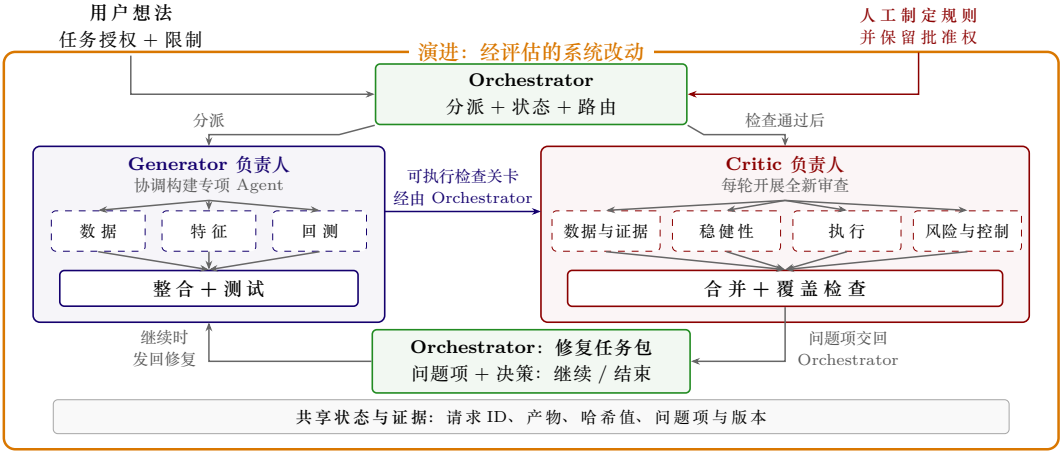

- 为每个版本记录测试时使用的模型、工具、数据、规则和提示词。
- 别名用于选择版本；批准仍需独立记录。
- 保留新旧两个版本及其证据。回滚会恢复先前的版本指针，并须在新案例中核验行为。

# 监测




- 监测反复出现的失败与共同风险。
- 模型、工具、数据源或权限变化时，重新评估。
- 将已记录的失败反馈给演进流程。提示词优化并不保证可靠性。

# 完整架构




- 定位角色 Harness、修订循环、交接路径与演进边界。

# 构建提纲

- 构建
  - 插件、角色与检查
  - 运行、修复与重放
  - 生产边界
- 演进
  - 评估改动、测试、批准或回滚

# 插件结构与入口

```
qt/
  kimi.plugin.json         # commands, skills, agents
  commands/{start,evo}.md  # two public entry points
  skills/{start,evo}/      # Orchestrator workflows
  agents/                  # two leads + six specialists
  references/              # schemas, role cards, policies
  runtime/quant_cn/        # state machine and numerical gates
  scripts/{qt,check}.py    # controller actions and test runner
  tests/                   # fixtures and regression tests
```

- 用 /qt:start 开展研究，用 /qt:evo 改进系统。
- 用 /plugins install <qt> 安装。修改源码后，重新安装并运行 /reload。
- 将安装后的代码与工作区 .qt/ 中的案例状态、提示词版本分开保存。

# 负责人与专项 Agent

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

- 配置负责人的工具权限及允许调用的专项 Agent。
- 检查实际分派与响应记录；仅凭报告文件，不能证明各角色使用了独立上下文。
- QT 使用 `qt-pit`（数据时序）、`qt-robustness` 和 `qt-execution`。独立风险审查属于设计扩展。

# Orchestrator 与检查关卡

```
reports, paths = load_team(output, response, request)
if state["role"] == "generator":
    generator_gate(output, state, response, round_dir)
else:
    merge_critic(state, reports, request)  # validate without mutating state
```

- Orchestrator 关联请求、证据与状态转换。
- Generator 报告与数值检查通过后，才能开展新的 Critic 审查。
- QT 要求三份 Generator 报告和三份 Critic 报告。
- 运行 `python3 <qt>/scripts/check.py` 检查控制器规则。

# Hook

```
{"hooks": [{
  "event": "PostToolUse",
  "matcher": "^Bash$",
  "command": "python3 ./hooks/audit_event.py",
  "timeout": 3
}]}
```

- 可选 Hook 练习（未随插件提供）：将 `audit_event.py` 复制到 `<qt>/hooks/`，合并清单配置，再重新安装并加载。
- 事件名、工具名及会话与调用 ID 只记录一次；它既不评估候选策略，也不记录每项专项 Agent 操作。
- Kimi 记录 Hook 错误或超时后会继续运行。必要检查仍由控制器执行。

# 运行一个研究案例

```
/qt:start Test monthly momentum on the bundled synthetic fixture

Inspect .qt/cases/<case>/:
  idea.json -> request.json -> rounds/round-001/
    generator/*.json -> numerical/ -> critic/*.json -> decision.json
  state.json (ledger), trace.jsonl, result.json (terminal round)
```

- 使用固定、带日期的输入；推进前检查时序与数值输出。
- 跟踪 `request_id`、报告、问题项和最终路由。
- 合成测试样例只改变动量的 `lookback` 和 `top_n`。
- 明确标注脚本演练；声称真实团队运行时须出示宿主分派记录。

# 修复提交

- 缺少报告或请求 ID 过期：否决提交，留在当前角色。
  - 第二次无效提交变为 `BLOCKED`。
- 不带 `--response` 调用 `advance`，再读取 `request.json`。
- 所有报告须匹配新 ID；核验证据并保留已封存产物。
- 问题：怎样修复，才能让这个临时测试案例继续?

举手回答

# 重放与恢复

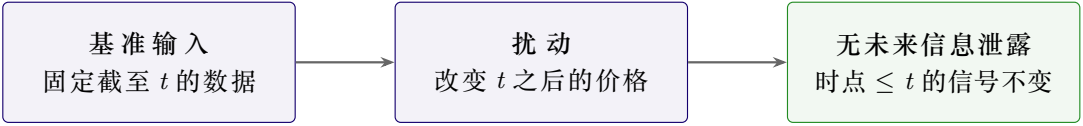
- 重建运行：输入/代码、问题项、修复、停止规则和角色卡版本。
- 固定输入与配置进行复算；新的模型评估使用新案例。
- 恢复待办工作前核验 ID 与哈希；已完成案例直接返回保存结果。
- 问题：案例本身或单独的记录是否标明了人工批准人?

举手回答

# 生产数据

- **输入与股票池**
  - 有许可证、带日期的数据；历史成分、停牌与退市。
- **执行**
  - 对应板块和日期的交易单位、价格限制、T+1 规则与费用。
- **运营**
  - 宿主隔离、指定责任人和独立测试的适配器。
- QT 使用五只虚构股票和合成工作日数据。
  生产数据、公司行为、组合控制与券商接入均待扩展。

# 研究验证




- 检验未来价格变动、数据重述、更高成本，以及依次推进的样本外区间表现。
- 通过这些检查，并不能证明策略将获得超额收益。

# 资金授权




- 检查组合限额、当前持仓、批准有效期与实际成交。
- 由指定负责人批准资金使用；研究验收通过不等于获得下单授权。
- `trade_sheet.csv` 使用独立的历史合成仓位计算样例，并非来自已验收的候选策略。QT 没有券商连接。

# 演进提纲

- 构建
  - 运行、修复并重放研究案例
- 演进
  - 记录失败并提出改动
  - 比较输出并检查回归
  - 版本管理、批准、回滚与监测

# 记录一次失败

```
python3 <qt>/scripts/qt.py evo-prepare --case case-01 --change change-01
```

- /qt:evo 开始系统改进；evo-prepare 关联已完成案例与变更前提示词版本。
- 修改前先记录失败及所需行为。
- 用开发案例获取反馈，另留案例用于最终评估。

# 提示词改动

```
  Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

- 问题：哪条指令针对了时序遗漏？需要什么证据才能说明它有效？
- 该 QT 接口只修改 Generator 提示词。策略参数、风险限额、验收规则与控制器代码均不在其修改范围内。

举手回答

# 比较输出

| 字段 | 基准版本 | 候选版本 |
| --- | --- | --- |
| 信号可得时间 | 29 Sep 15:00 | 29 Sep 15:00 |
| 执行时间 | 29 Sep 15:00 | 30 Sep 09:30 |
| 证据 ID | `PRICE_01` | `MISSING_02` |

- 交易从 30 Sep 09:30 (UTC+8) 起才允许执行。仅留存了 `PRICE_01`。
- 运行配套检查，或在 promptfoo 中加载 `promptfooconfig.json`，比较两份输出。
- 问题：是否有任一输出同时通过两项检查?

举手回答

# 判断证据

| 检查项 | 基准版本 | 候选版本 |
| --- | --- | --- |
| 执行时序 | 未通过 | 通过 |
| 证据引用 | 通过 | 未通过 |
| 采用决定 | 否决 | 否决 |

- 候选版本修正了时序，却引用了缺失的来源 `MISSING_02`。
- 问题：采用该版本前，还需修复哪些问题，并进行哪些独立检查？

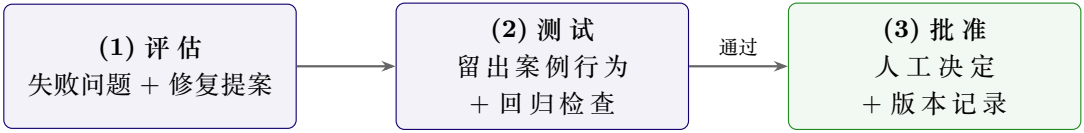
举手回答

# 用 Git 管理版本




- 提交审查后的代码、提示词与技能，将 commit ID 关联到测试证据和配置。
- 为已批准版本打标签，便于恢复。
- QT 注册表选择当前提示词；将它与 Git 关联仍是扩展功能。
- 标签记录版本，批准仍需独立记录。

# 评估、测试、批准

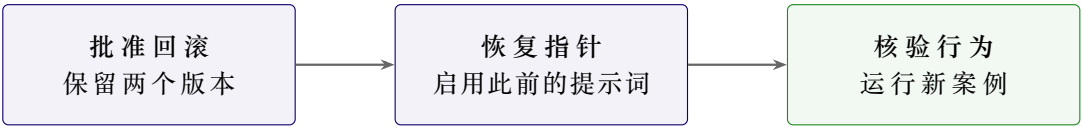



评估或测试未通过：保留当前版本。

- 围绕记录的失败评估改动，并测试留出案例的行为与既有控制。
- QT 的 `evo-evaluate` 运行回归测试并核验审查字段，之后才进入 `AWAITING_APPROVAL`。
- 明确的人工批准才允许 `evo-adopt`；否决则保留当前版本。
- 问题：已保存的证据是否足以支持批准?

举手回答

# 回滚与监测




- 获得批准后，用 `evo-rollback` 恢复此前的指针。保留新旧两个版本及其证据。
- 已有案例保留原配置，运行中途不切换版本。
- 在新案例中核验恢复后的版本，再监测后续运行中是否反复出现失败。

# 最终审查

- **证据**：追溯并复现已记录的工作。
- **控制**：要求未满足时阻断或解释。
- **连续性**：恢复已核验工作，避免重复不确定操作。
- **改进**：评估并恢复按规则管理的系统版本。
  - 问题：选择一项控制。它回应谁的需要，哪些内容已测试，还有什么需要构建？

举手回答